

Automatización de Topologías de Red en GNS3 Mediante Inteligencia Artificial y Model Context Protocol

Andrés Felipe García del Río (afgarciadelrio@ucompensar.edu.co) - Jose Luis Castro Martin
(jluiscastro@ucompensar.edu.co) – Gabriel Enrique de Hoyos Ozuna
(gdehoyos@ucompensar.edu.co)
Ingeniería en Telecomunicaciones

Resumen

Este trabajo presenta el desarrollo de GNS3 AI Studio, una plataforma basada en inteligencia artificial capaz de automatizar la creación y configuración de topologías de red en GNS3 mediante descripciones en lenguaje natural. El sistema integra el protocolo Model Context Protocol (MCP), la API de Anthropic Claude Sonnet, la API REST de GNS3 y un backend desarrollado con FastAPI. La solución permite generar automáticamente dispositivos, enlaces, direccionamiento IP y configuraciones de protocolos como OSPF. Los resultados obtenidos demuestran una reducción significativa en el tiempo requerido para desplegar laboratorios de redes, pasando de varias horas de configuración manual a pocos minutos mediante automatización inteligente.

I. Introducción

La automatización de redes se ha convertido en un elemento fundamental dentro de los entornos modernos de telecomunicaciones y ciberseguridad. Los laboratorios de simulación permiten validar configuraciones y protocolos antes de su implementación en ambientes productivos. Sin embargo, la creación manual de topologías complejas en plataformas como GNS3 requiere conocimientos técnicos avanzados y un tiempo considerable de configuración.

Con el avance de los modelos de inteligencia artificial generativa y protocolos de integración como Model Context Protocol (MCP), es posible desarrollar soluciones capaces de interpretar instrucciones en lenguaje natural y traducirlas automáticamente en configuraciones técnicas [1]. En este contexto, se desarrolló GNS3 AI Studio, una plataforma que integra inteligencia artificial con GNS3 para automatizar la creación de topologías de red y la configuración de routers Cisco IOS.

II. Metodología

El desarrollo del sistema se realizó sobre Ubuntu 26.04 LTS utilizando Python 3.14 como lenguaje principal. La arquitectura implementada integra cuatro componentes fundamentales:

1. API REST de GNS3 para la administración de proyectos y dispositivos.
2. Model Context Protocol (MCP) como mecanismo de comunicación entre la IA y las herramientas del sistema.
3. API de Anthropic Claude Sonnet para el procesamiento de instrucciones en lenguaje natural.
4. Backend FastAPI con comunicación WebSocket para interacción en tiempo real.

El sistema utiliza dispositivos Cisco IOSv, IOSvL2, VPCS y switches Ethernet integrados en GNS3. Las

configuraciones de routers se realizan mediante conexiones Telnet asíncronas utilizando la librería telnetlib3. Además, se implementó una interfaz web tipo ChatGPT desarrollada con HTML, CSS y JavaScript.

El servidor MCP fue diseñado con 14 herramientas funcionales, entre ellas: creación de proyectos, agregado de nodos, conexión de dispositivos, inicio y detención de routers, y configuración automática de protocolos OSPF.

Paso a Paso de la Instalación

Paso 1 — Preparar Ubuntu

```
bash
```

```
sudo apt update
```

```
sudo apt upgrade -y
```

Paso 2 — Instalar GNS3

```
bash
```

```
sudo add-apt-repository ppa:gns3/ppa
```

```
sudo apt update
```

```
sudo apt install gns3-gui gns3-server -y
```

```
sudo usermod -aG ubridge,libvirt,kvm,wireshark $USER
```

```
sudo reboot
```

Durante la instalación se aceptan los diálogos sobre permisos de usuarios no privilegiados.

Paso 3 — Configurar GNS3

Abrir GNS3 y completar el Setup Wizard:

- **Server type:** Local
- **Host:** 127.0.0.1
- **Port:** 3080

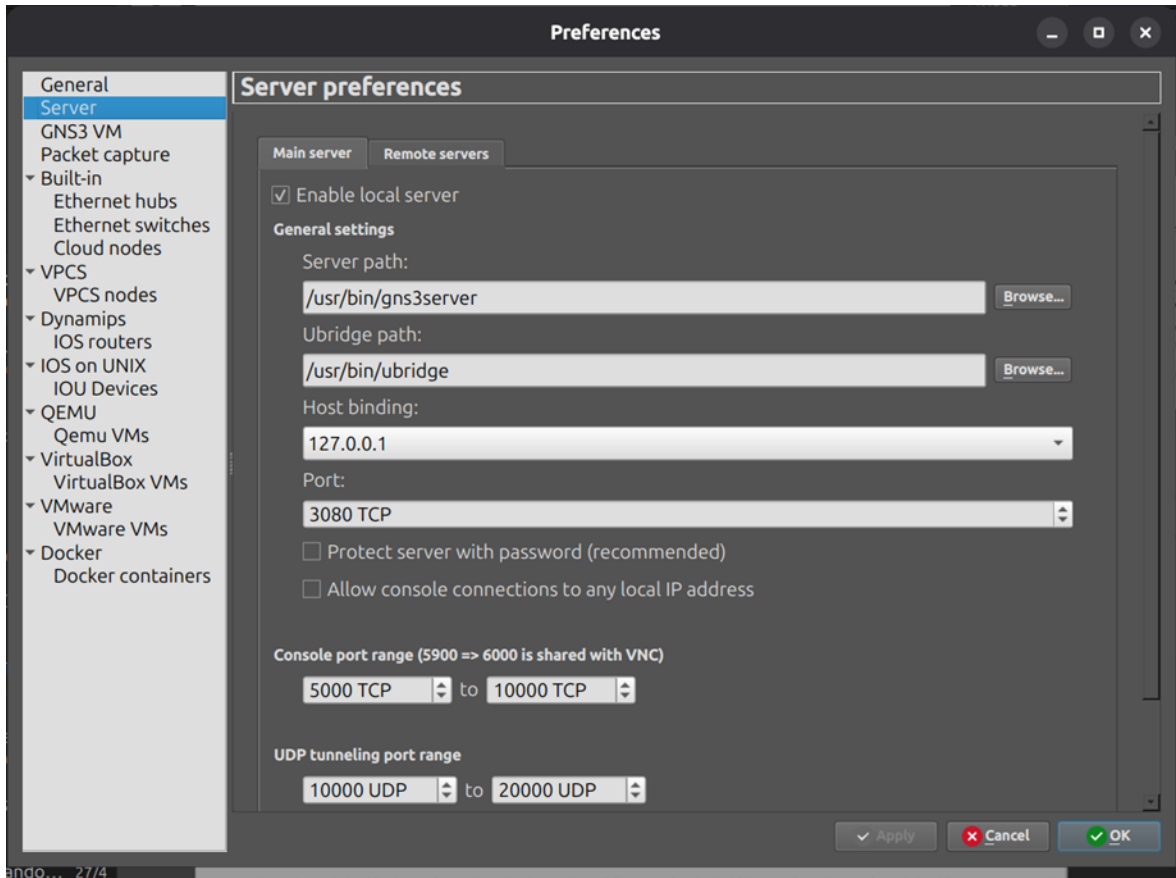


Ilustración 1 Configuración de IOS en GNS3

Verificar que la API REST responde:

```
bash
```

```
curl http://localhost:3080/v2/version
```

Paso 4 — Instalar Python y dependencias del sistema

```
bash
```

```
sudo apt install python3 python3-pip python3-venv git curl wget nodejs npm -y
```

```
python3 --version # Verificar 3.12+
```

```
gabriel@gabriel-ThinkBook-14-G6-IRL:~/gns3-mcp$ python3 --version
Python 3.14.4
gabriel@gabriel-ThinkBook-14-G6-IRL:~/gns3-mcp$
```

Ilustración 2 confirmación de servicios en python

```
bash
```

```
mkdir -p ~/gns3-mcp/plantillas
```

```
cd ~/gns3-mcp
```

```
python3 -m venv venv
```

```
source venv/bin/activate
```

```
pip install requests anthropic mcp telnetlib3 pyyaml fastapi uvicorn websockets httpx
```

Paso 6 — Obtener IDs de las plantillas de GNS3

Se creó un script auxiliar para descubrir dinámicamente los `template_id` que GNS3 asigna a cada dispositivo:

```
python
```

```
# obtener_templates.py
```

```
import requests
```

```
r = requests.get("http://localhost:3080/v2/templates")
```

```
for t in r.json():
```

```
    print(f'{t["name"]:<40} {t["template_id"]}')"
```

Paso 7 — Crear archivos del proyecto

Se desarrollaron 4 archivos Python principales:

1. **cisco_config.py** — Módulo asíncrono que se conecta por Telnet a los routers Cisco IOSv y envía comandos de configuración (hostname, IPs, OSPF, etc.)
2. **gns3_helpers.py** — Capa de abstracción sobre la API REST de GNS3. Maneja:
 - Creación/eliminación de proyectos
 - Agregar nodos (routers, switches, PCs)
 - Crear enlaces entre nodos
 - Iniciar/detener dispositivos
 - Búsqueda dinámica de templates
3. **gns3_mcp_server.py** — Servidor MCP que expone 14 herramientas al modelo Claude vía JSON-Schema:
 - `listar_proyectos`, `listar_nodos`
 - `crear_proyecto`, `eliminar_proyecto`
 - `agregar_nodo`, `conectar_nodos`

- iniciar_nodos, detener_nodos
 - configurar_router, ejecutar_comandos
 - crear_lan_simple, crear_topologia_hub_spoke
 - crear_topologia_triangulo, crear_topologia_lineal
4. **app.py** — Backend FastAPI con WebSocket que orquesta toda la comunicación entre el navegador, Claude y el MCP.

Paso 8 — Obtener API Key de Anthropic

1. Crear cuenta en console.anthropic.com
2. Generar API Key en Settings → API Keys → Create Key
3. Agregar saldo (\$5 USD para pruebas)

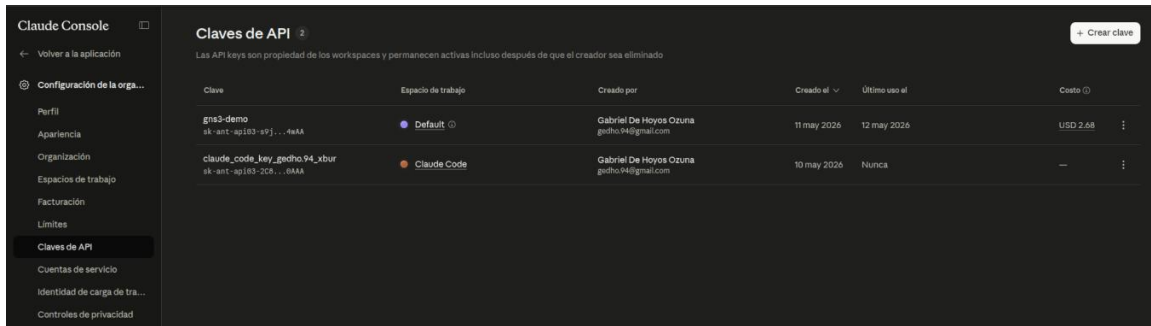


Ilustración 3 verificación de Saldo e inicio de pruebas

4. Configurar en Ubuntu:

bash

```
echo 'export ANTHROPIC_API_KEY="sk-ant-..." >> ~/.bashrc
```

```
source ~/.bashrc
```

Paso 9 — Crear interfaz web

Se desarrolló una interfaz HTML/CSS/JS con:

- Diseño tipo ChatGPT (oscuro, moderno)
- Sidebar con estado del sistema en tiempo real
- Botones de acciones rápidas
- Chat con burbujas diferenciadas para usuario/IA
- Tarjetas visuales para cada herramienta ejecutada
- Indicador de "escribiendo..." animado
- WebSocket para comunicación bidireccional

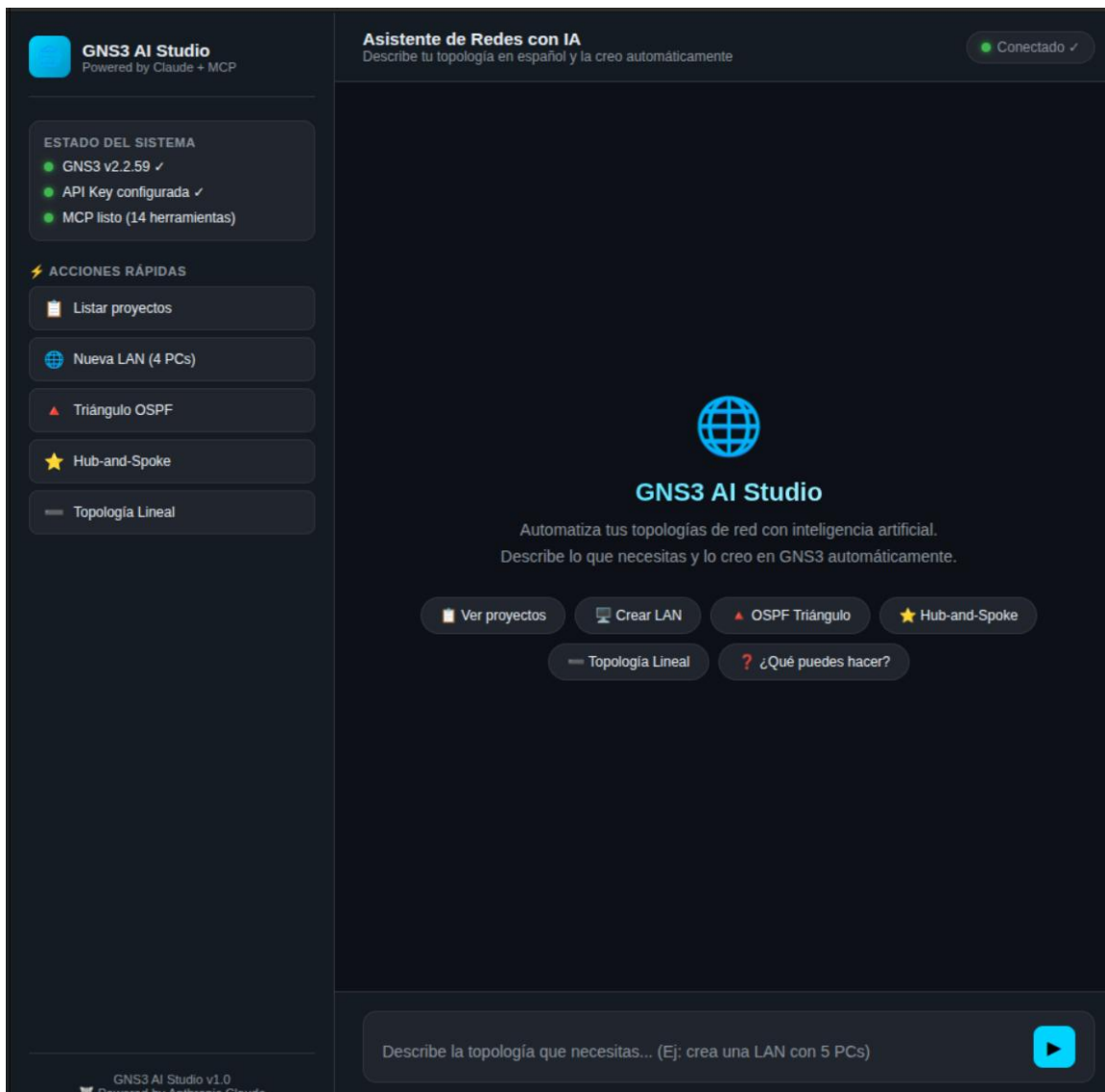


Ilustración 4 Version física y en vivo del proyecto

Paso 10 — Empaquetar como .deb

Se desarrolló un script `build_deb.sh` que:

- Crea estructura de paquete Debian
- Empaqueta el código + venv con todas las dependencias
- Agrega script de primera configuración (zenity)
- Crea icono y entry en menú de aplicaciones
- Genera el archivo .deb instalable con doble clic

III. Resultados

Las pruebas realizadas permitieron validar la correcta automatización de múltiples escenarios de red. Entre las topologías implementadas exitosamente se encuentran redes LAN, configuraciones lineales, topologías triangulares OSPF y arquitecturas hub-and-spoke.

El sistema logró crear y configurar topologías completas en aproximadamente tres minutos, incluyendo direccionamiento IP, establecimiento de adyacencias OSPF y verificación de rutas. Además, se verificó la interoperabilidad con dispositivos open source como VyOS, MikroTik y pfSense.

Las métricas obtenidas indican una reducción significativa del tiempo de implementación. Mientras que una topología OSPF compleja puede requerir entre 2 y 4 horas de configuración manual, la plataforma desarrollada reduce este proceso a pocos minutos mediante automatización basada en IA.

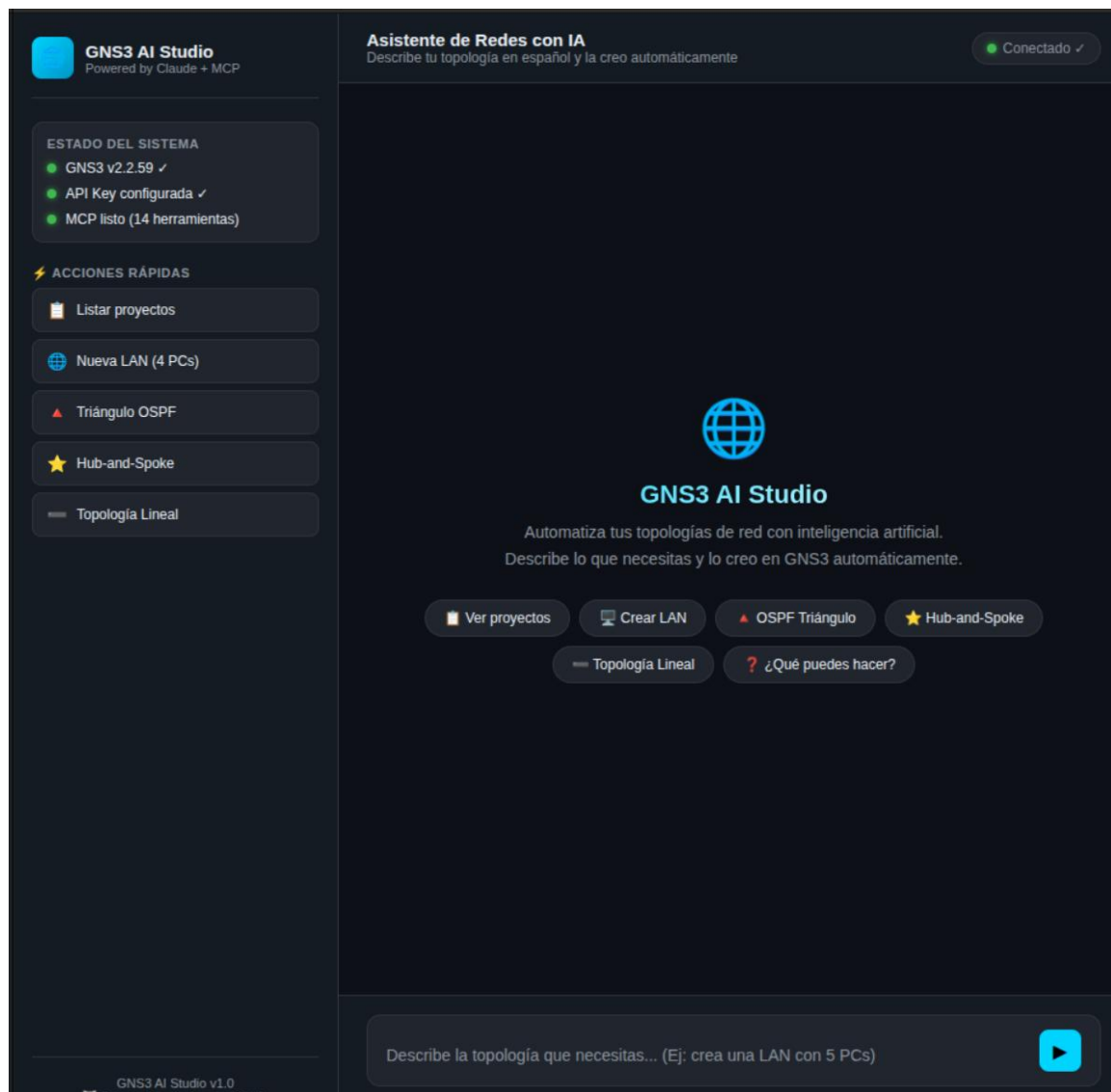


Ilustración 5 version final de la herramienta

IV. Discusión

Durante el desarrollo del proyecto se identificaron múltiples desafíos técnicos relacionados con la integración entre GNS3, MCP y los modelos de inteligencia artificial. Uno de los principales problemas fue la limitación de tiempo de espera de Claude Desktop, lo que obligó al desarrollo de un cliente Python personalizado para la interacción directa con la API de Anthropic.

También se presentaron dificultades relacionadas con la administración dinámica de templates en GNS3, debido a que los identificadores UUID cambian entre instalaciones. Para solucionar este inconveniente se implementó un mecanismo de descubrimiento dinámico de templates mediante consultas a la API REST.

Otro aspecto relevante fue la distribución del sistema a usuarios finales. La solución consistió en la creación de un instalador .deb que automatiza la instalación de dependencias, el entorno virtual de Python y la configuración inicial del sistema.

V. Conclusiones

El proyecto GNS3 AI Studio demuestra que la integración de inteligencia artificial con plataformas de simulación de redes permite automatizar procesos complejos de configuración y despliegue. La utilización de Model Context Protocol y modelos de lenguaje avanzados facilita la interacción en lenguaje natural con entornos de laboratorio de redes.

Los resultados obtenidos evidencian mejoras importantes en eficiencia operativa, reducción de tiempo y simplificación de tareas repetitivas. Además, la arquitectura desarrollada permite futuras expansiones hacia protocolos avanzados como BGP, EIGRP, VLANs y soporte multi-fabricante.

La solución propuesta representa una herramienta con alto potencial de aplicación en universidades, centros de entrenamiento CCNA/CCNP, departamentos de redes y centros de operaciones NOC.

Estructura de Archivos del Proyecto

~/gns3-mcp/

```
|— venv/          # Entorno virtual Python aislado
|— cisco_config.py  # Configuración de routers vía Telnet
|— gns3_helpers.py  # Funciones helper de la API de GNS3
|— gns3_mcp_server.py  # Servidor MCP con 14 herramientas
|— gns3_marketplace.py  # Integración con marketplace GNS3
|— chat_gns3.py     # Cliente de chat en terminal (backup)
|— app.py           # Backend FastAPI (web)
|— static/
|  |— index.html     # Interfaz web tipo ChatGPT
```



```
|— start.sh          # Script de inicio
|— build_deb.sh      # Generador del paquete .deb
```

Declaración sobre el Uso de IA:

En el desarrollo del presente trabajo se utilizaron herramientas de Inteligencia Artificial para asistencia en la redacción técnica, corrección gramatical, organización estructural del manuscrito y apoyo en la adaptación del contenido al formato académico IEEE. Asimismo, se emplearon modelos de IA como soporte en el análisis y documentación del proyecto relacionado con la automatización de topologías de red en GNS3 mediante Model Context Protocol (MCP). Todos los resultados, configuraciones, validaciones técnicas y conclusiones fueron revisados, verificados y editados manualmente por los autores antes de su presentación final.

Declaración de Responsabilidad de los Autores:

Los autores asumen plena responsabilidad sobre la exactitud, originalidad e integridad del contenido presentado en este manuscrito, incluyendo el diseño, implementación, pruebas y documentación del sistema GNS3 AI Studio. Asimismo, garantizan que el uso de herramientas asistidas por Inteligencia Artificial no reemplazó el criterio técnico ni la validación académica realizada durante el desarrollo del proyecto.

Palabras Clave

GNS3, MCP, Inteligencia Artificial, Automatización de Redes, OSPF, FastAPI, Anthropic Claude.

Referencias

- [1] Anthropic, “Model Context Protocol Documentation, (<https://modelcontextprotocol.io>)” 2025.
- [2] GNS3 Technologies Inc., “GNS3 REST API Documentation, (<https://gns3-server.readthedocs.io/en/latest/>)” 2025.
- [3] FastAPI, “FastAPI Framework Documentation, (<https://fastapi.tiangolo.com>)” 2025.
- [4] Anthropic, “Claude API Reference (<https://docs.anthropic.com>),” 2025.
- [5] Python Software Foundation, “Python 3.14 Documentation,” 2025.
- [6] GNS3 Marketplace, “GNS3 Appliances and Templates, (<https://www.gns3.com/marketplace/appliances>)” 2025.